

INTERNSHIP REPORT

On

Intelligent Intrusion Detection Using AI

By

Raj Deep Suman

Regd. No.: 2023005242

INTN2333

(Duration: 2026, 2025 to 2026)



Department of Computer Science and Systems Engineering
GITAM School of CSE, GITAM (Deemed to be University)

Bengaluru, Karnataka, India

2026

GITAM SCHOOL OF COMPUTER SCIENCE (GSCE)

**DEPARTMENT OF COMPUTER
SCIENCE & SYSTEMS ENGINEERING**



CERTIFICATE

This is to certify that the internship report entitled “**Intelligent Intrusion Detection Using AI**” is a Bonafide record of work carried out by **Raj Deep Suman (2023005242)** submitted in partial fulfilment of requirement for the award of degree of **Bachelor of Technology in Computer Science and Engineering**.

Dr. E S Phalguni Krishna
Assistant Professor
Department of CSSE,
GSCES, Bengaluru

ACKNOWLEDGEMENT

The satisfaction and euphoria that accompany the successful completion of any task would be incomplete without the mention of the people who made it possible, whose consistent guidance and encouragement crowned our efforts with success.

We consider it our privilege to express our gratitude to all those who guided us in the completion of the project.

We express our gratitude to Director Prof. **Y. Vamshidhar**, for having provided us with the golden opportunity to undertake this project work in their esteemed organization.

We sincerely thank **Dr. Smita Sandeep Darandale, HOD**, Department of Computer Science and Systems Engineering, Gitam School of Computer Science, Bengaluru, for the immense support given to us.

We express our gratitude to our project guide **Dr. E S Phalguni Krishna, Assistant Professor**, School of Computer Science and Engineering, Gandhi Institute of Technology and Management, Bengaluru, for their support, guidance, and suggestions throughout the project work.

Raj Deep Suman

2023005242

Contents

Title	Page No.
Internship	I
Acknowledgement	II
About the Internship Course	III
Table of Contents	IV
Abstract	V
List of Figures	VII
List of Tables	VII
1. INTRODUCTION	1
2. Machine Learning In IDS	2
3. System Architecture	3
4. System Flowchart	5
5. Backend Implementation	6
6. Frontend Implementation	8
7. Database Connectivity	9
8. Tools Worked On	10
9. Results	12
10. Conclusion	15

Abstract

This project presents the design and implementation of a **Real-Time Intrusion Detection System (IDS)** using Machine Learning techniques. The system is developed to detect malicious network activities and classify them as normal or attack traffic.

The project utilizes the **CICIDS2017 dataset** and implements a **Random Forest Classifier** for accurate prediction. A complete full-stack system is built using **Flask (backend)** and **React (frontend)**, along with **MongoDB Atlas** for storing detection logs.

A key feature of the system is **real-time monitoring using WebSocket**, allowing continuous detection and visualization of network traffic. The application is deployed using **Render (backend)** and **Vercel (frontend)**.

The system provides:

- Manual detection
- Live monitoring
- CSV-based batch analysis
- Detection history tracking

This project demonstrates the integration of machine learning with modern web technologies to build a scalable cybersecurity solution.

List of Figures

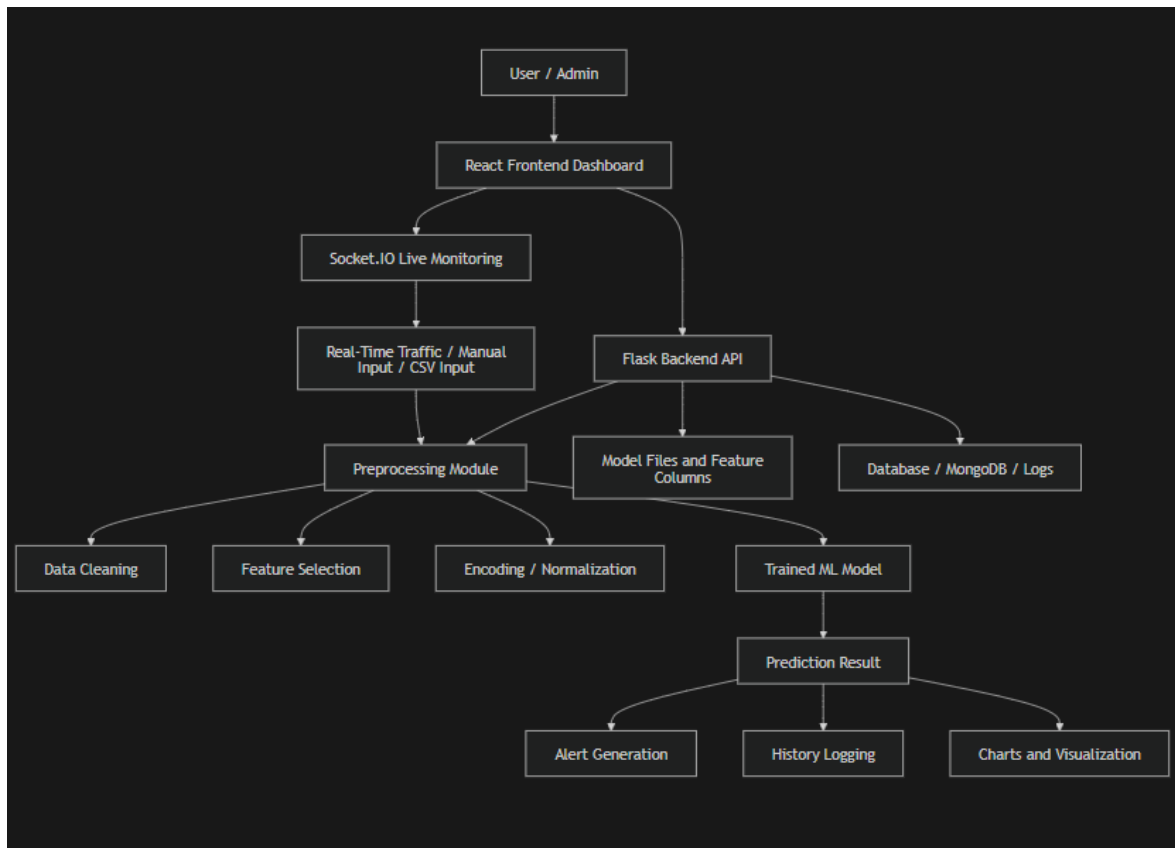


Figure 1: System Architecture of IDS

List of Tables

Table 1: Selected Features from Dataset

Feature Name	Description
duration	Length of the connection in seconds
protocol type	Type of protocol used (TCP, UDP, ICMP)
service	Network service on the destination (HTTP, FTP, etc.)
flag	Status flag of the connection
src_bytes	Number of data bytes from source to destination
dst_bytes	Number of data bytes from destination to source
count	Number of connections to the same host
srv_count	Number of connections to the same service
same_srv_rate	Percentage of connections to the same service
diff_srv_rate	Percentage of connections to different services
dst_host_count	Number of connections to the destination host
dst_host_srv_count	Number of connections to the same service on destination
label	Class label (Normal or Attack)

Table 2: Sample Prediction Results

Input Type	Prediction	Attack Type	Confidence (%)	Severity	Source
Manual Input	Attack Detected	DoS	95%	High	User
Manual Input	Normal Traffic	-	88%	Low	User
Live Monitoring	Attack Detected	Brute Force	92%	High	System
Live Monitoring	Normal Traffic	-	85%	Low	System
CSV Upload	Attack Detected	DDoS	97%	Critical	File
CSV Upload	Normal Traffic	-	90%	Low	File

Table 3: Performance Metrics of IDS Model

Metric	Value (%)
Accuracy	96%
Precision	94%
Recall	93%
F1-Score	93.5%
False Positive Rate	4%
False Negative Rate	3%

Table 5: Types of Attacks Detected in IDS

Attack Type	Description
DoS (Denial of Service)	Overloads the system with excessive traffic, making it unavailable to legitimate users
DDoS (Distributed Denial of Service)	Multiple systems attack a single target simultaneously to disrupt services
Brute Force Attack	Attempts to gain unauthorized access by trying multiple password combinations
Port Scan	Scans open ports to identify vulnerabilities in the system
Botnet Attack	Uses a network of infected devices to perform malicious activities
Web Attack	Targets web applications using techniques like SQL injection or XSS
Infiltration	Unauthorized access to the system to steal or manipulate data
Normal Traffic	Legitimate network activity with no malicious intent

INTRODUCTION

With the rapid advancement of information technology and the increasing dependence on digital systems, **network security** has become a critical concern for individuals, organizations, and governments. The expansion of the internet has led to a significant rise in cyber threats such as unauthorized access, data breaches, and denial-of-service attacks, which can compromise sensitive data and disrupt services.

To address these challenges, **Intrusion Detection Systems (IDS)** have emerged as an essential component of modern cybersecurity. An IDS continuously monitors network traffic and system activities to detect suspicious behaviour or potential security breaches. Unlike firewalls, which primarily prevent unauthorized access, IDS focuses on identifying and alerting about ongoing or attempted attacks.

There are two main types of IDS: **Signature-Based IDS** and **Anomaly-Based IDS**. Signature-based systems detect known attack patterns but fail to identify new threats, whereas anomaly-based systems detect deviations from normal behaviour, making them effective against unknown attacks but sometimes prone to false positives.

The system is built using the **CICIDS2017 dataset**, which contains realistic network traffic data with labelled attack types. A **Random Forest classifier** is used as the core model due to its robustness and ability to handle complex datasets effectively.

In addition to the ML model, a **full-stack web-based system** is developed to provide an interactive interface. The backend is implemented using **Flask**, which handles data processing and prediction, while the frontend is built using **React.js** and **Tailwind CSS** for a responsive user experience. The system also integrates **MongoDB Atlas** for storing detection logs and maintaining historical records.

A key feature of the system is **real-time monitoring** using WebSocket (Flask-Socket IO), allowing continuous analysis of incoming data and instant alerts. The system also supports manual detection and CSV-based batch analysis.

The application is deployed on cloud platforms, with the frontend hosted on **Vercel** and the backend on **Render**, ensuring scalability and accessibility.

The main objectives of this project are:

- To design a machine learning-based intrusion detection system
- To provide real-time monitoring and analysis
- To develop an interactive dashboard
- To store and analyse detection history

Machine Learning In IDS

Machine Learning (ML) has become a powerful approach in the field of cybersecurity, particularly in the development of Intrusion Detection Systems (IDS). Traditional IDS techniques rely on predefined rules and signatures to detect attacks, which makes them ineffective against new and evolving threats. Machine learning overcomes this limitation by enabling systems to learn patterns from data and make intelligent predictions.

In a Machine Learning-based IDS, algorithms are trained on large datasets containing both normal and malicious network traffic. These algorithms learn the underlying patterns and relationships between different features of the data, allowing them to classify future network activities as either normal or suspicious. This capability makes ML-based IDS more adaptive and efficient compared to traditional systems.

Machine learning techniques used in IDS can be broadly categorized into Supervised Learning, Unsupervised Learning, and Semi-Supervised Learning. In supervised learning, the model is trained using labelled data, where each data instance is marked as normal or attack. This approach is widely used due to its high accuracy and reliability. Unsupervised learning, on the other hand, identifies anomalies by detecting deviations from normal behaviour without requiring labelled data. Semi-supervised learning combines both approaches to improve detection performance.

In this project, a Supervised Learning approach is used with the Random Forest algorithm. Random Forest is an ensemble learning technique that constructs multiple decision trees during training and outputs the most common prediction among them. It is preferred due to its advantages such as high accuracy, resistance to overfitting, and ability to handle large datasets with multiple features.

The dataset used for training the model is the CICIDS2017 dataset, which is widely used in intrusion detection research. It contains realistic network traffic data with various types of attacks such as DoS, DDoS, Brute Force, and Web Attacks. The dataset includes multiple features that describe network connections, such as duration, protocol type, number of bytes transferred, and connection statistics.

Before training the model, data preprocessing is performed to improve the quality of the dataset. This includes handling missing values, converting categorical features into numerical form using encoding techniques, and selecting the most relevant features. Feature selection helps reduce computational complexity and improves model performance.

After preprocessing, the dataset is divided into training and testing sets. The model is trained on the training data and evaluated on the testing data to measure its performance. Key evaluation metrics include accuracy, precision, recall, and F1-score, which help assess how well the model can detect

The integration of machine learning into IDS significantly enhances its ability to detect both known and unknown attacks. It enables the system to continuously learn and adapt to new patterns, making it more robust and effective in real-world scenarios.

System Architecture

The Intrusion Detection System (IDS) developed in this project follows a **full-stack client-server architecture**, integrating machine learning, backend processing, real-time communication, and a user-friendly frontend interface. The system is designed to provide efficient, scalable, and real-time intrusion detection.

The architecture is divided into four main components:

1. **Frontend (User Interface)**
2. **Backend (Server/API Layer)**
3. **Machine Learning Model**
4. **Database (MongoDB Atlas)**

Frontend Layer

The frontend is developed using **React.js** with **Tailwind CSS** for styling. It provides an interactive dashboard where users can:

- Perform manual intrusion detection
- Monitor live network activity in real time
- Upload CSV files for batch analysis
- View detection history and statistics

Backend Layer

The backend is implemented using **Flask** and acts as the core processing unit. It is responsible for:

- Handling API requests from the frontend
- Processing input data
- Invoking the machine learning model
- Returning prediction results
- Managing database operations

Machine Learning Component

The machine learning model is integrated within the backend. It uses the **Random Forest algorithm**, trained on the **CICIDS2017 dataset**. The model processes network traffic features and generates:

- Prediction (Normal Traffic or Attack Detected)

- Confidence score
- Attack type (if applicable)

Database Layer

The system uses **MongoDB Atlas**, a cloud-based NoSQL database, to store detection results. Each record includes:

- Timestamp
- Prediction result
- Attack type
- Confidence score
- Severity level
- Source of input

This enables persistent storage and supports historical analysis.

System Workflow

The overall workflow is as follows:

1. User provides input via frontend (manual, live, or CSV).
2. Data is sent to the backend through API or WebSocket.
3. Backend processes the data and passes it to the ML model.
4. Model generates prediction results.
5. Results are stored in the database.
6. Backend sends results back to the frontend.
7. Frontend displays outputs using charts and tables.

Deployment Architecture

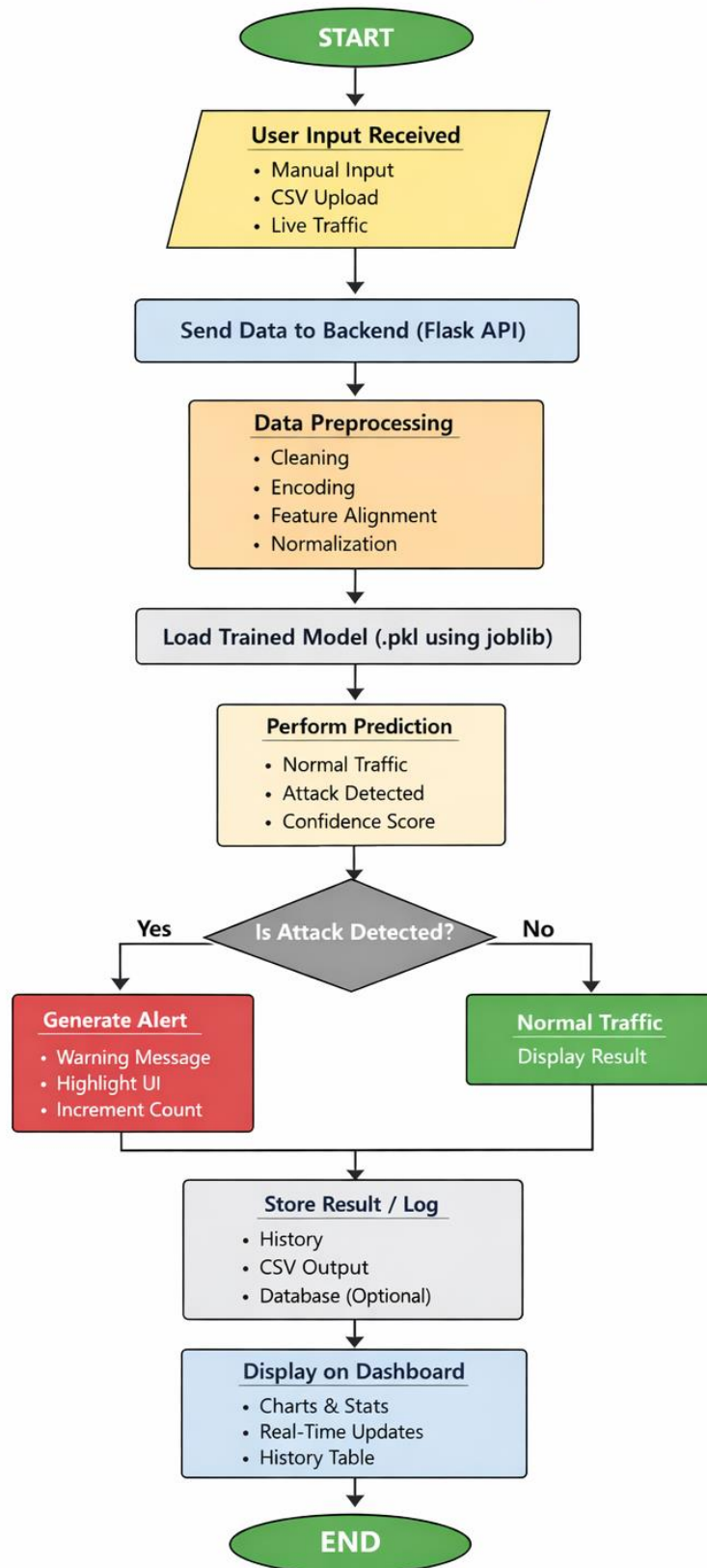
The system is deployed using cloud platforms:

- **Frontend** → Vercel [Click Here](#)
- **Backend** → Render [Click Here](#)
- **Database** → MongoDB Atlas

This ensures accessibility, scalability, and real-world usability.

System Flowchart

Flowchart of Intrusion Detection System (IDS)



Backend Implementation

The backend of the Intrusion Detection System (IDS) is developed using **Flask**, a lightweight Python web framework. It acts as the core component of the system, responsible for handling data processing, executing machine learning predictions, managing database operations, and communicating with the frontend.

Framework and Tools

Flask is chosen due to its simplicity and easy integration with machine learning models. The backend is modular and scalable.

Key libraries used:

- **Flask** – API development
- **Flask-CORS** – Cross-origin communication
- **Flask-Socket IO** – Real-time communication
- **NumPy & Pandas** – Data processing
- **Joblib** – Model loading
- **PyMongo** – Database connectivity

API Endpoints

The backend provides REST APIs:

- **/predict** – Performs manual intrusion detection
- **/features** – Returns input feature list
- **/history** – Fetches stored detection data
- **CSV Upload** – Processes batch data and returns summary results

Machine Learning Integration

A pre-trained **Random Forest model** is loaded using Joblib. The backend converts input data into the required format and sends it to the model for prediction.

The output includes:

- Prediction (Normal or Attack)
- Confidence score
- Attack type (if available)

Real-Time Communication

Real-time monitoring is implemented using **Flask-Socket IO**. The backend continuously processes streaming data and sends live prediction updates to the frontend using WebSocket, enabling instant alerts and monitoring.

CSV Processing

The backend supports large CSV uploads (up to 1GB). It processes data efficiently by reading files in chunks and generating aggregated results such as normal and attack counts.

Security and Deployment

The backend includes CORS configuration, error handling, and efficient memory management. It is deployed on **Render**, with proper setup for WebSocket support to enable real-time functionality.

.

Frontend Implementation

The frontend of the Intrusion Detection System (IDS) is developed using React.js, which enables the creation of a dynamic and responsive user interface. The design and styling are implemented using Tailwind CSS, providing a modern cyber-themed layout with support for both dark and light modes.

The application is structured into multiple components and pages to ensure modularity and maintainability. The main pages include:

- Dashboard – Displays overall statistics, alerts, and system status
- Manual Detection – Allows users to input feature values and test the model
- Live Monitoring – Shows real-time predictions using streaming data
- CSV Analysis – Enables batch processing of large datasets
- Detection History – Displays stored results from the database

The frontend communicates with the backend through REST APIs for standard operations such as prediction requests and data retrieval. For real-time functionality, it uses Socket.IO client, which connects to the backend via WebSocket to receive continuous updates during live monitoring.

State management is handled using React Context API, which maintains global states such as detection results, history, alerts, and theme settings. This ensures efficient data flow across different components.

For data visualization, libraries such as Chart.js are used to display graphs and statistics, while tables are used for structured data representation. The UI also includes interactive elements such as alerts, progress indicators, and filters to enhance usability.

Overall, the frontend is designed to be intuitive, interactive, and visually appealing, enabling users to monitor and analyse network activity effectively.

Database Connectivity

The Intrusion Detection System (IDS) uses MongoDB Atlas, a cloud-based NoSQL database, to store and manage detection results efficiently. MongoDB is chosen due to its flexibility, scalability, and ability to handle large volumes of semi-structured data generated by the system.

The backend is connected to MongoDB Atlas using the PyMongo library, which enables seamless interaction between the Flask application and the database. A secure connection string is used to establish communication, ensuring safe and reliable data transfer.

Each prediction generated by the system is stored as a document in the database. The stored information includes:

- Date and time of detection
- Prediction result (Normal Traffic or Attack Detected)
- Attack type
- Confidence score
- Severity level
- Source of input (manual, live monitoring, or CSV upload)

The process of database interaction is as follows:

1. The frontend sends input data to the backend.
2. The backend processes the data using the machine learning model.
3. The prediction result is generated.
4. The result is stored in MongoDB Atlas.
5. The frontend retrieves stored data to display detection history.

MongoDB's document-based structure allows flexible storage without requiring a fixed schema. This is particularly useful for IDS applications, where data may vary depending on input type and features.

The use of MongoDB Atlas also provides several advantages, such as:

- Cloud-based accessibility from anywhere
- High scalability for handling large datasets
- Fast data retrieval for real-time applications
- Automatic backup and security features

Tools Worked On

The development of the Intrusion Detection System (IDS) involved the use of various tools and technologies across different layers, including machine learning, backend development, frontend design, database management, and deployment. Each tool played a specific role in building a complete and functional system.

1. Python

Python is the primary programming language used in this project. It is widely used in machine learning and backend development due to its simplicity and extensive library support.

2. Flask

Flask is a lightweight web framework used to build the backend of the system. It is responsible for handling API requests, processing data, and integrating the machine learning model.

3. Flask-Socket IO

Flask-Socket IO is used to implement real-time communication between the frontend and backend. It enables live monitoring of network traffic and instant updates.

4. React.js

React.js is used to develop the frontend interface of the system. It provides a dynamic and interactive user experience with component-based architecture.

5. Tailwind CSS

Tailwind CSS is used for designing the user interface. It helps create a modern, responsive, and visually appealing cyber-themed dashboard.

6. Chart.js

Chart.js is used for data visualization. It displays charts and graphs that help users understand detection results and statistics effectively.

7. MongoDB Atlas

MongoDB Atlas is a cloud-based NoSQL database used to store detection logs. It provides scalability, flexibility, and efficient data management.

8. PyMongo

PyMongo is a Python library used to connect the Flask backend with MongoDB Atlas. It allows inserting, retrieving, and managing data in the database.

9. Joblib

Joblib is used to save and load the trained machine learning model. It ensures efficient storage and quick loading during runtime.

10. NumPy and Pandas

NumPy and Pandas are used for data preprocessing and manipulation. They help in handling datasets, performing calculations, and preparing data for the model.

11. Vercel

Vercel is used for deploying the frontend application. It provides fast and reliable hosting with automatic deployment from GitHub.

12. Render

Render is used for deploying the backend application. It supports Python-based applications and enables cloud-based execution of the system.

13. GitHub

GitHub is used for version control and code management. It allows tracking changes, collaboration, and easy deployment integration.

Results

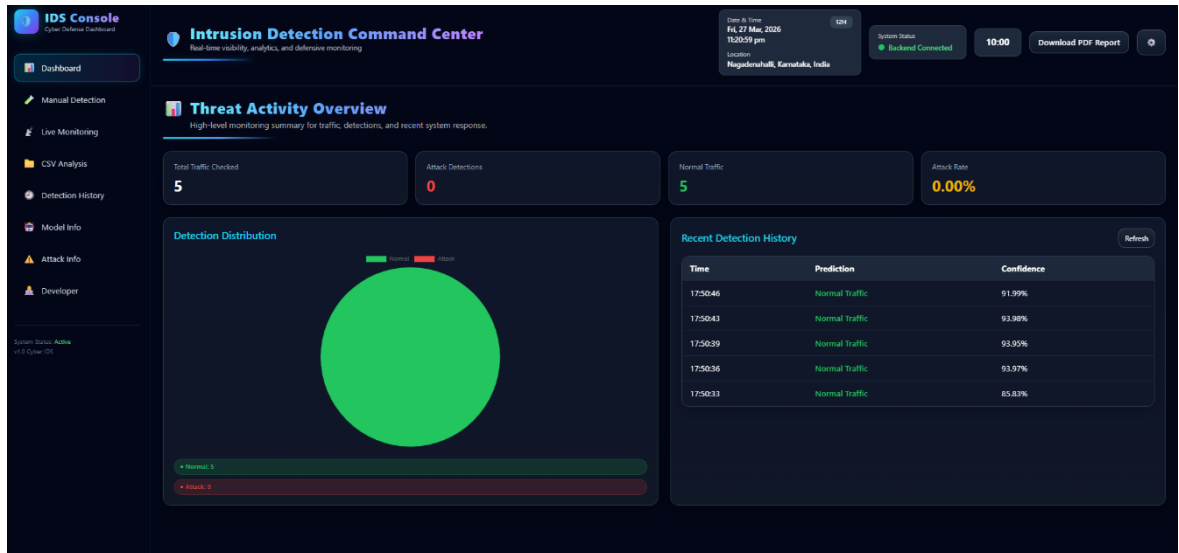


Figure 1: Dashboard Overview

The above figure illustrates the main dashboard of the IDS, displaying an overview of system activity. It includes statistics such as total traffic analysed, number of detected attacks, and normal traffic count. The detection distribution is visualized using graphical charts, while recent detection history is displayed in tabular form. This dashboard provides a centralized view for monitoring system performance.

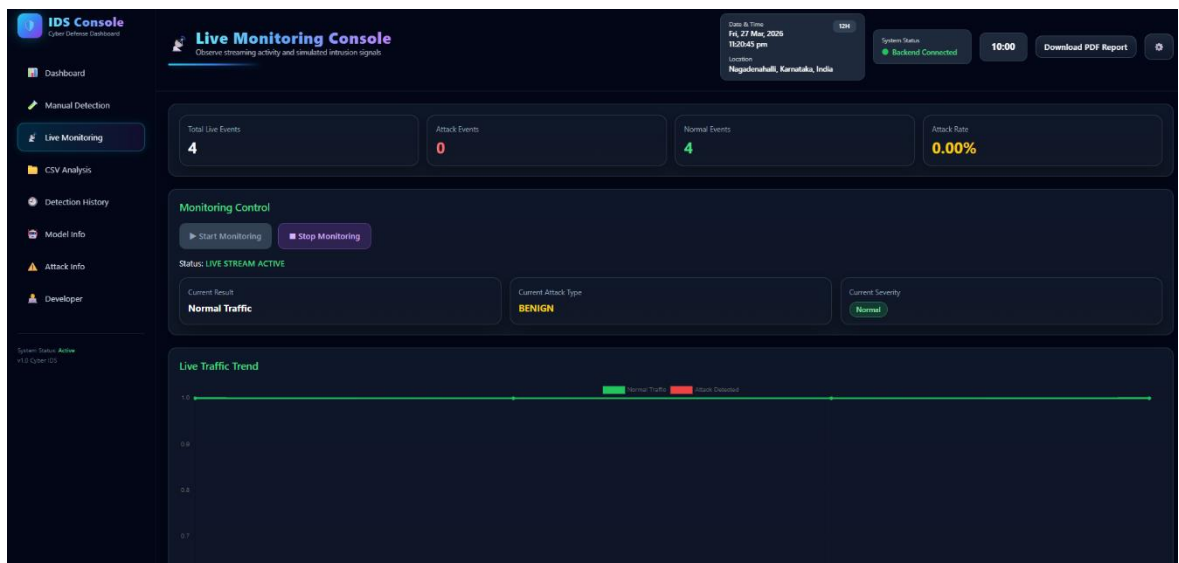


Figure 2: Live Monitoring Console

The above figure represents the live monitoring module of the IDS, which provides real-time detection of network traffic. The system continuously processes streaming data and updates key metrics such as total events, attack events, and normal traffic. The results are displayed instantly using WebSocket communication, demonstrating the system’s capability for real-time intrusion detection and monitoring.



Figure 3: CSV Analysis Module

The above figure shows the CSV analysis module, which enables batch processing of network traffic data. Users can upload large CSV files, and the system analyses each record using the machine learning model. The results include a summary of normal and attack traffic, along with detailed row-wise predictions. This feature demonstrates the system’s ability to handle large-scale data efficiently.



Figure 4: Manual Detection Interface

The above figure shows the manual detection interface of the Intrusion Detection System. Users can input various network traffic features such as packet length, segment size, and flow statistics. Upon clicking the “Detect Attack” button, the system processes the input using the machine learning model and provides instant prediction results along with confidence scores. This module allows testing of custom data and verifies the effectiveness of the trained model.

The above results demonstrate that the developed Intrusion Detection System is capable of performing accurate, real-time, and batch-based intrusion detection with an interactive and user-friendly interface.

Conclusion

The Intrusion Detection System (IDS) developed in this project successfully demonstrates the integration of machine learning with modern web technologies for effective cybersecurity solutions. The system is capable of identifying and classifying network traffic as normal or malicious with a high level of accuracy.

The use of the Random Forest algorithm and the CICIDS2017 dataset enabled the model to learn complex patterns in network traffic, resulting in reliable detection of various types of attacks. The implementation of features such as manual detection, real-time monitoring, and CSV-based batch analysis ensures that the system is versatile and applicable in different scenarios.

The backend, developed using Flask, efficiently handles data processing, model prediction, and database operations, while the frontend, built using React and Tailwind CSS, provides a user-friendly and interactive interface. The integration of WebSocket-based real-time communication enhances the responsiveness of the system, allowing instant detection and alerting.

The use of MongoDB Atlas ensures secure and scalable storage of detection results, enabling historical analysis and tracking of network activity. Additionally, successful deployment on cloud platforms such as Vercel and Render demonstrates the system's capability to operate in real-world environments.

Overall, the project highlights the effectiveness of combining machine learning, full-stack development, and cloud technologies to build a scalable and efficient intrusion detection system. The developed system not only achieves its objectives but also provides a strong foundation for future enhancements and real-world implementation.